

COMPLETE REFERENCE

CSS Selectors

The Definitive Visual Guide

Basic · Combinators · Pseudo-classes · Pseudo-elements · Attribute · Specificity

7

Categories

40+

Selectors

Examples

Every Type

■ TABLE OF CONTENTS

1. **Basic Selectors** — Universal, Type, Class, ID
2. **Combinator Selectors** — Descendant, Child, Adjacent, General Sibling
3. **Pseudo-class Selectors** — Structural, User-action, Form, Logical
4. **Pseudo-element Selectors** — ::before, ::after, ::first-line, and more
5. **Attribute Selectors** — Exact, Prefix, Suffix, Contains, and more
6. **Specificity & Cascade** — How browsers decide which styles win
7. **Quick-Reference Cheat Sheet** — Every selector at a glance

1 · BASIC SELECTORS

Basic selectors are the foundation of CSS. They target elements directly by their tag name, class attribute, ID attribute, or apply universally to all elements.

Selector	Syntax	Description
Universal	*	Selects every element in the document.
Type	element	Selects all elements with a given tag name, e.g. p , div , h1 .
Class	.className	Selects all elements that have the specified class attribute.
ID	#idName	Selects the single element with the matching id attribute (must be unique).
Multiple	A, B	Groups selectors — applies styles to elements matched by A or B.

Basic Selectors — Examples

```
/* Universal selector */
* { box-sizing: border-box; margin: 0; }

/* Type selector */
h1 { font-size: 2rem; color: #1e293b; }

/* Class selector */
.card { border-radius: 8px; padding: 16px; }

/* ID selector */
#navbar { position: sticky; top: 0; }

/* Grouped selectors */
h1, h2, h3 { font-family: 'Inter', sans-serif; }
```

2 · COMBINATOR SELECTORS

Combinators define the relationship between selectors. They allow you to target elements based on their position relative to other elements in the HTML tree.

Selector	Syntax	Description
Descendant	<code>A B</code>	Selects B that is anywhere inside A (any depth).
Child	<code>A > B</code>	Selects B that is a direct child of A only.
Adjacent Sibling	<code>A + B</code>	Selects the first B immediately after A (same parent).
General Sibling	<code>A ~ B</code>	Selects all B elements that follow A (same parent).
Column	<code>A B</code>	Selects B belonging to a table column A (CSS Grid).

Combinator Selectors — Examples

```
/* Descendant — any depth */
nav a { text-decoration: none; color: inherit; }

/* Child — direct only */
ul > li { list-style: disc; }

/* Adjacent sibling — first next element */
h2 + p { font-size: 1.1rem; font-weight: 500; }

/* General sibling — all following */
h2 ~ p { color: #64748b; }
```

A BDescendant
(any depth)**A > B**Child
(direct only)**A + B**Adjacent
(next sibling)**A ~ B**General
(all following)

3 · PSEUDO-CLASS SELECTORS

Pseudo-classes apply styles based on an element's state or position in the document tree. They are prefixed with a single colon `:`.

3a · User-Action States

Selector	Syntax	Description
<code>:hover</code>	<code>a:hover</code>	Mouse pointer is over the element.
<code>:focus</code>	<code>input:focus</code>	Element has keyboard/programmatic focus.
<code>:active</code>	<code>btn:active</code>	Element is being activated (clicked).
<code>:visited</code>	<code>a:visited</code>	Link has been visited by the user.
<code>:link</code>	<code>a:link</code>	Unvisited link.
<code>:focus-within</code>	<code>div:focus-within</code>	Element or a descendant has focus.
<code>:focus-visible</code>	<code>input:focus-visible</code>	Focus is visible (keyboard navigation).

3b · Structural / Position

Selector	Syntax	Description
<code>:first-child</code>	<code>li:first-child</code>	First child of its parent.
<code>:last-child</code>	<code>li:last-child</code>	Last child of its parent.
<code>:nth-child(n)</code>	<code>li:nth-child(2)</code>	nth child — accepts integers, keywords, or $An+B$.
<code>:nth-last-child(n)</code>	<code>li:nth-last-child(2)</code>	nth child counting from the end.
<code>:only-child</code>	<code>p:only-child</code>	Element with no siblings.
<code>:first-of-type</code>	<code>p:first-of-type</code>	First of its tag type among siblings.
<code>:last-of-type</code>	<code>p:last-of-type</code>	Last of its tag type among siblings.
<code>:nth-of-type(n)</code>	<code>p:nth-of-type(odd)</code>	nth of its type — accepts $An+B$, odd, even.
<code>:only-of-type</code>	<code>img:only-of-type</code>	Only element of its type among siblings.
<code>:root</code>	<code>:root</code>	Highest-level parent (usually <code><html></code>).
<code>:empty</code>	<code>div:empty</code>	Element with no children or text.

3c · Form / Input States

Selector	Syntax	Description
<code>:checked</code>	<code>input:checked</code>	Checkbox or radio button is checked.
<code>:disabled</code>	<code>input:disabled</code>	Form element is disabled.

<code>:enabled</code>	<code>input:enabled</code>	Form element is enabled.
<code>:required</code>	<code>input:required</code>	Has the required attribute.
<code>:optional</code>	<code>input:optional</code>	Does not have the required attribute.
<code>:valid</code>	<code>input:valid</code>	Value passes its validation constraint.
<code>:invalid</code>	<code>input:invalid</code>	Value fails its validation constraint.
<code>:in-range</code>	<code>input:in-range</code>	Value is within the specified min/max range.
<code>:out-of-range</code>	<code>input:out-of-range</code>	Value is outside the specified min/max range.
<code>:placeholder-shown</code>	<code>input:placeholder-shown</code>	Placeholder text is currently visible.
<code>:read-only</code>	<code>input:read-only</code>	Element is not editable.
<code>:read-write</code>	<code>input:read-write</code>	Element is editable.
<code>:indeterminate</code>	<code>input:indeterminate</code>	Checkbox in an indeterminate state.

3d · Logical Pseudo-classes

Selector	Syntax	Description
<code>:is()</code>	<code>:is(h1, h2, h3)</code>	Matches any of the given selectors (takes highest specificity).
<code>:not()</code>	<code>p:not(.special)</code>	Negation — matches if the selector does NOT match.
<code>:has()</code>	<code>div:has(> img)</code>	Matches if descendants/siblings satisfy the argument.
<code>:where()</code>	<code>where(header, footer) a</code>	Like <code>:is()</code> but contributes zero specificity.
<code>:any-link</code>	<code>a:any-link</code>	Matches <code>:link</code> and <code>:visited</code> combined.
<code>:target</code>	<code>#section:target</code>	Element whose id matches the URL fragment (#).
<code>:lang()</code>	<code>p:lang(fr)</code>	Matches element with a specific language attribute.
<code>:scope</code>	<code>:scope > div</code>	Element that is the stylesheet's scope (or <code>:root</code>).
<code>:local-link</code>	<code>a:local-link</code>	Links pointing to the same domain.

Pseudo-class Examples

```
/* Hover & focus states */
button:hover { background: #3b82f6; transform: translateY(-1px); }
input:focus { outline: 2px solid #38bdf8; border-color: transparent; }

/* Structural – zebra striping */
tr:nth-child(even) { background: #f8fafc; }
tr:first-child { font-weight: 700; background: #0f172a; color: #fff; }

/* Logical – :is() shorthand */
:is(h1, h2, h3) { font-family: 'Poppins', sans-serif; }

/* Form validation */
input:valid { border-color: #34d399; }
input:invalid { border-color: #f87171; }

/* :has() – parent selector */
div:has(> img) { display: grid; place-items: center; }
```

4 · PSEUDO-ELEMENT SELECTORS

Pseudo-elements create virtual elements that aren't in the HTML source. They use the double-colon `::` prefix (single colon is legacy, still works for `::before/::after`).

Selector	Syntax	Description
<code>::before</code>	<code>p::before</code>	Inserts generated content before the element's content.
<code>::after</code>	<code>p::after</code>	Inserts generated content after the element's content.
<code>::first-line</code>	<code>p::first-line</code>	Styles the first formatted line of a block element.
<code>::first-letter</code>	<code>p::first-letter</code>	Styles the first letter (drop-cap effect).
<code>::selection</code>	<code>p::selection</code>	Styles text that the user has highlighted/selected.
<code>::placeholder</code>	<code>input::placeholder</code>	Styles the placeholder text inside an input.
<code>::marker</code>	<code>li::marker</code>	Styles the list item marker (bullet or number).
<code>::backdrop</code>	<code>dialog::backdrop</code>	Full-screen layer behind a dialog or fullscreen element.
<code>::file-selector-button</code>	<code>input[type=file]::file-selector-button</code>	The 'choose file' button.
<code>::cue</code>	<code>video::cue</code>	Styles WebVTT cues (subtitle/caption text).
<code>::spelling-error</code>	<code>::spelling-error</code>	Browser-identified spelling error text.
<code>::grammar-error</code>	<code>::grammar-error</code>	Browser-identified grammar error text.

Pseudo-element Examples

```
/* Decorative ::before / ::after */
.card::before { content: ''; display: block;
                height: 4px; background: linear-gradient(90deg,
                #38bdf8, #818cf8); border-radius: 4px 4px 0 0; }

/* Drop-cap with ::first-letter */
article p:first-of-type::first-letter {
    font-size: 3.5rem; float: left;
    line-height: 1; margin-right: 6px; color: #0f172a; }

/* Custom selection colour */
::selection { background: #38bdf8; color: #0f172a; }

/* Styled list markers */
ul li::marker { color: #818cf8; font-weight: 700; }

/* Placeholder styling */
input::placeholder { color: #94a3b8; font-style: italic; }
```


5 · ATTRIBUTE SELECTORS

Attribute selectors match elements based on the presence or value of HTML attributes. They are wrapped in square brackets `[]`.

Selector	Syntax	Description
<code>[attr]</code>	<code>[href]</code>	Has the attribute, regardless of value.
<code>[attr=val]</code>	<code>[type="text"]</code>	Attribute equals exactly val.
<code>[attr~=val]</code>	<code>[class~="card"]</code>	Attribute contains val as a whitespace-separated word.
<code>[attr =val]</code>	<code>[lang "en"]</code>	Attribute is exactly val or starts with val-.
<code>[attr^=val]</code>	<code>[href^="https"]</code>	Attribute starts with val.
<code>[attr\$=val]</code>	<code>[href\$=".pdf"]</code>	Attribute ends with val.
<code>[attr*=val]</code>	<code>[href*="docs"]</code>	Attribute contains val anywhere.
<code>[attr=val i]</code>	<code>[type="TEXT" i]</code>	Case-insensitive match (add i flag).
<code>[attr=val s]</code>	<code>[data-id="A" s]</code>	Case-sensitive match (add s flag, default).

Attribute Selector Examples

```
/* Any element with a href attribute */
[href] { cursor: pointer; }

/* Exact match */
input[type="submit"] { background: #3b82f6; color: #fff; }

/* Starts with – external links */
[href^="https://"]::after { content: " ■"; font-size: 0.75em; }

/* Ends with – file type icons */
[href$=".pdf"] { background-image: url(pdf-icon.svg); }
[href$=".xlsx"] { background-image: url(xlsx-icon.svg); }

/* Contains – partial match */
[class*="col-"] { padding: 0 12px; }

/* Language – starts with or equals */
[lang|"zh"] { font-family: "Noto Sans SC", sans-serif; }
```

6 · SPECIFICITY & THE CASCADE

When multiple CSS rules target the same element, the browser uses the cascade to decide which styles win. The three main factors are: **Specificity**, **Order of appearance**, and **Importance**.

Specificity Weight Table

Selector Type	Column	Weight	Example
Inline styles	A (1,0,0,0)	1000	style="color:red"
ID selectors	B (0,1,0,0)	100	#navbar
Class / Pseudo-class / Attribute	C (0,0,1,0)	10	.card :hover [href]
Type / Pseudo-element	D (0,0,0,1)	1	div ::before
Universal / Combinators	—	0	* > + ~

Specificity in Action

```
/* Specificity examples */
p          { color: black; } /* 0,0,0,1 = 1 */
.text      { color: blue;  } /* 0,0,1,0 = 10 */
#main .text p { color: green; } /* 0,1,1,1 = 111 */

/* !important overrides everything (use sparingly) */
.error { color: red !important; }

/* :where() contributes 0 specificity — great for resets */
:where(h1,h2,h3) { margin: 0; } /* 0,0,0,0 = 0 */

/* :is() takes the specificity of its highest argument */
:is(#id, .cls) { color: teal; } /* 0,1,0,0 = 100 (from #id) */
```

■ **Tip:** Calculate specificity as (A, B, C, D) and compare left to right. Avoid **!important** — prefer increasing specificity or restructuring selectors.

7 · QUICK-REFERENCE CHEAT SHEET

Basic	
*	All elements
div	Tag type
.class	Class
#id	ID
A, B	A or B

Pseudo-classes (state)	
:hover	Mouse over
:focus	Has focus
:active	Being clicked
:visited	Link visited
:checked	Checked input
:disabled	Disabled
:valid	Valid value
:invalid	Invalid value

Logical Pseudo-classes	
:is(A, B)	Matches A or B
:not(A)	Does not match A
:has(A)	Has descendant A
:where(A, B)	Like :is(), 0 specificity

Combinators	
A B	B inside A
A > B	Direct child B of A
A + B	B right after A
A ~ B	All B after A

Pseudo-classes (structural)	
:first-child	First child
:last-child	Last child
:nth-child(n)	nth child
:nth-child(odd)	Odd children
:nth-child(even)	Even children
:only-child	No siblings
:first-of-type	First of type
:empty	No children
:root	Root element
:target	URL fragment match

Pseudo-elements	
::before	Generated content before
::after	Generated content after
::first-line	First line of block
::first-letter	First letter
::selection	Highlighted text
::placeholder	Placeholder text
::marker	List item bullet

Attribute	
[attr]	Has attribute
[attr="v"]	Equals value
[attr^="v"]	Starts with
[attr\$="v"]	Ends with
[attr*="v"]	Contains
[attr~="v"]	Word in list
[attr = "v"]	Equals or val-

■ Remember: Specificity order: **Inline** > **ID** > **Class/Pseudo/Attr** > **Type/Pseudo-element** > **Universal**